Databricks + Neo4j

The Better Together Value

What the Graph Looks Like

- Graphs naturally model the real world
- Data in Neo4j lives as **nodes** (entities/nouns) and **relationships** (how they connect)
- In the diagram: `(parentheses)` are nodes, `[:brackets]` are relationships

```
(Aircraft) -[:HAS_SYSTEM]-> (System) -[:HAS_COMPONENT]-> (Component)
     |
     |--[:OPERATED_FLIGHT]-> (Flight) -[:DEPARTED_FROM]-> (Airport)
                                  |
                                  |--[:HAD_DELAY]-> (Delay)
```

Every node and relationship can carry properties (names, dates, measurements), so the graph is rich with context — not just connections.

Why Combine Databricks and Neo4j?

Databricks and Neo4j solve **different problems well**.

Databricks excels at working with large volumes of structured data — aggregations, time-series analysis, and machine learning over tables.

Neo4j excels at understanding **how things connect** — following chains of relationships, finding patterns, and answering questions about structure.

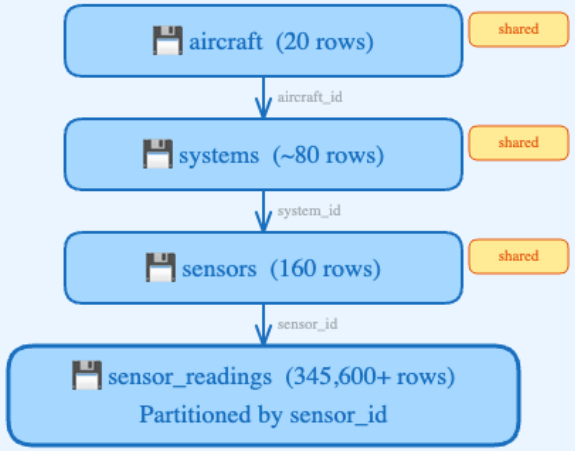
Most real-world problems have both: numbers that need crunching **and** relationships that need navigating. Using both platforms together gives you the best of each.

Aircraft Digital Twin — Dual Database Architecture

20 aircraft | 160 sensors | 345,600+ readings | 800 flights | 300 maintenance events

📊 Databricks Lakehouse

Time-Series Analytics + SQL



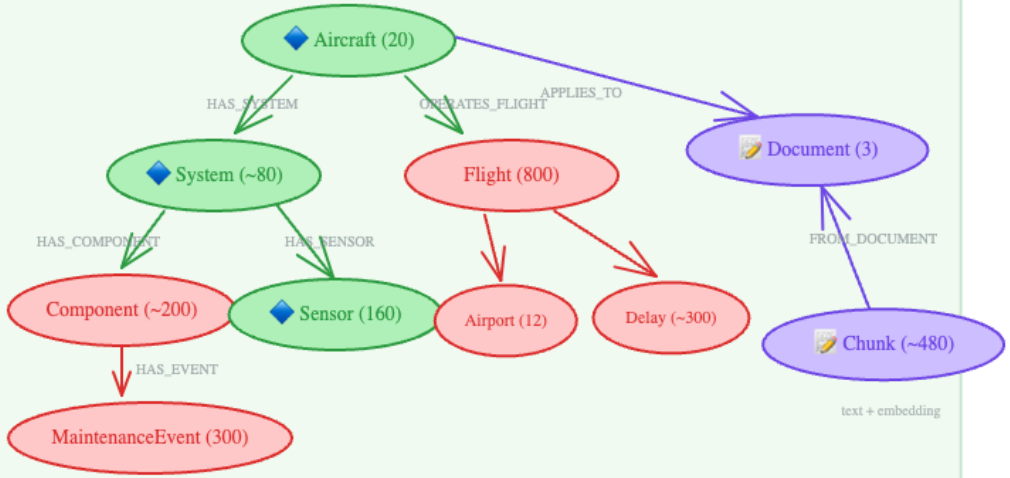
🎯 Optimized For:

- Time-series aggregations (AVG, STDDEV, PERCENTILE)
- Hourly / daily / weekly trend analysis
- Anomaly detection (readings above P95)
- Fleet-wide statistical comparisons
- Rolling window calculations
- Natural language queries via Genie
- SQL-based reporting & dashboards

```
SELECT AVG(r.value), STDDEV(r.value)
FROM sensor_readings r
  JOIN sensors sen ON r.sensor_id = sen.sensor_id
WHERE sen.type = 'EGT'
```

🔷 Neo4j Aura (Graph)

Topology + Relationships + Cypher



- Shared with Lakehouse
- Neo4j only (graph-exclusive)
- GraphRAG / Semantic Search

🎯 Optimized For:

- Multi-hop relationship traversals (no JOINs)
- Aircraft topology: Aircraft > System > Component > Sensor
- Maintenance-to-delay root cause analysis
- Flight routing and airport network patterns
- Vector + fulltext search on maintenance manuals
- GraphRAG retrieval with entity extraction

```
MATCH (a:Aircraft)-[:HAS_SYSTEM]->(s:System)
    -[:HAS_COMPONENT]->(c:Component)
    -[:HAS_EVENT]->(m:MaintenanceEvent)
RETURN a.tail_number, c.name, m.fault, m.severity
```

Building a Knowledge Graph from Lakehouse Data

The typical pattern is:

1. **Store your raw data** in Databricks — CSVs, Delta tables, whatever you have
2. **Use the Neo4j Connector** to write that data into Neo4j as nodes and relationships
3. **Not all data moves** — only the subset that fits a graph model (entities and relationships) is extracted
4. **Query the graph** to answer relationship-heavy questions that tables struggle with

Example: An aircraft fleet dataset becomes a connected graph where you can naturally ask "which components were on aircraft that had delays after a specific type of maintenance?" — a question that would require many table joins but is a simple graph traversal.

GraphRAG: Smarter Document Search

GraphRAG combines document search with graph context to give AI better answers.

How it works:

1. **Break documents into chunks** — split maintenance manuals into smaller pieces
2. **Generate embeddings** — use Databricks Foundation Models to create vector representations
3. **Store chunks in Neo4j** — each chunk becomes a node, linked to the next chunk and to the document it came from
4. **Connect documents to the graph** — link maintenance manuals to the aircraft, systems, and components they describe

Now when someone searches for "how to troubleshoot engine vibration," the system finds relevant text **and** knows which aircraft and components are involved.

GraphRAG: Adding Graph Context to Vector Search

Standard vector search finds text that is **semantically similar** to your question. That is a great starting point.

GraphRAG builds on that foundation by adding **graph context**:

Approach	What You Get
Vector search alone	"Here are text chunks about engine vibration"
GraphRAG	"Here are text chunks about engine vibration, **plus** the affected aircraft, their maintenance history, and related component data"

The graph connections turn an isolated text answer into a **contextual, grounded response**.

Flexible Vector Store Options

GraphRAG's vector store is **pluggable** — you are not locked into a single vector database.

The `neo4j-graphrag-python` library supports **external vector stores** through its `ExternalRetriever` pattern. Vectors live in the store your team already uses, while Neo4j provides the graph context enrichment.

Vector Store	How It Works
Neo4j (built-in)	Vectors and graph in one database — simplest setup
Databricks Vector Search	Keep vectors in your Lakehouse alongside your Delta tables
Pinecone, Weaviate, Qdrant	Built-in support via optional extras in `neo4j-graphrag-python`

The pattern: The external store handles similarity search and returns matching IDs. Neo4j resolves those IDs to nodes and traverses the graph for context. Each system does what it is best at.

The key insight: Graph context is the value-add — regardless of where your vectors live.

Three Ways to Retrieve Knowledge

GraphRAG supports multiple retrieval strategies depending on how much context you need:

Vector Search — Find the most relevant text by meaning
Good for general questions like "what causes hydraulic pressure loss?"

Graph-Enhanced Search — Find relevant text, then follow graph connections
Good for specific questions like "what maintenance is required for this aircraft's fuel system?"

Hybrid Search — Combine meaning-based and keyword-based search
Good for domain-specific terms and fault codes that need exact matching

Neo4j in Unity Catalog: Federated Queries via JDBC

Neo4j can also be registered as a **JDBC connection in Unity Catalog**, making graph data queryable alongside your Lakehouse tables using standard SQL.

How it works:

- A JDBC connection to Neo4j is configured in Unity Catalog with the Neo4j JDBC driver
- The driver **automatically translates SQL into Cypher** — so users and tools write SQL, and Neo4j executes it as a graph query
- Graph data can be queried directly or **materialized as Delta tables** for use in dashboards and Genie spaces

Why it matters:

- **Unified governance** — graph data is managed under the same Unity Catalog access controls and lineage as everything else
- **Federated queries** — combine graph relationships and Lakehouse metrics in a single SQL statement

Neo4j as an MCP Server

Model Context Protocol (MCP) is an open standard that lets AI agents use external tools. Neo4j can act as an MCP server, giving any AI agent the ability to query a knowledge graph.

What the Neo4j MCP server exposes:

- **Get the schema** — the agent learns what types of nodes and relationships exist
- **Run read queries** — the agent can ask questions of the graph in Cypher

This means an AI agent on Databricks can **explore and query Neo4j on its own** — it discovers the shape of your data, then writes and executes queries to answer user questions. No pre-built integrations needed.

The Data Discovery Problem

- Enterprise lakehouses grow fast — **hundreds of schemas, thousands of tables**
- Analysts spend significantly more time finding and understanding data than actually analyzing it
- AI agents can write SQL, but struggle to know **which table to query** or **what a column means**
- Unity Catalog governs access, but treats relationships as join paths — not **first-class semantic connections**
- The graph can fill this gap: a **semantic layer** that maps business meaning onto physical data

Building a Semantic Layer

- **Sync Unity Catalog metadata into Neo4j**: catalogs, schemas, tables, columns as a connected graph
- Layer on **domain knowledge**: business concepts mapped to physical assets, metric definitions, authoritative sources
- **Metadata enrichment**: domain experts curate the semantic connections — no data duplication
- Neo4j makes these connections **traversable** — follow paths from business concepts to tables, metrics to sources, and entities across domains

Neo4j as a Semantic Layer

- Analysts discover data faster through **business concepts** instead of hunting through schemas
- Agents use **GraphRAG patterns on metadata** — semantic search for a business concept, then traverse the graph to find related schemas, tables, and columns
- Agents get **structured business context** — know what to query, not just how to query
- Text-to-SQL accuracy jumps when agents understand the domain
- **Complementary to Unity Catalog**: UC governs access and lineage, the graph adds **meaning and connections**

Multi-Agent Analytics

- Data lives in **both** platforms — build specialized AI agents for each
- Instead of one agent that tries to do everything, create **focused agents** that each do one thing well
- A **supervisor** routes questions to the right agent automatically
- The end user asks a question in plain English — the system figures out where to go

The Genie Space Agent

The first agent lives entirely in Databricks and is built using **Genie Space**.

What it does: Answers questions about sensor readings and telemetry — anything that lives in Lakehouse tables.

How it works: You connect Genie to your Unity Catalog tables, give it domain knowledge about your data (what the columns mean, what normal ranges look like), and it translates natural language into SQL.

Good at:

- "What is the average engine temperature this month?"
- "Show vibration trends over the last 30 days"
- "Compare fuel flow between Boeing and Airbus aircraft"

The Neo4j MCP Agent

The second agent connects to Neo4j through the **MCP server** registered in Unity Catalog.

What it does: Answers questions about relationships, structure, and operational history — anything that lives in the knowledge graph.

How it works: The agent discovers the graph schema through MCP, then generates and runs Cypher queries to answer questions. It understands how things are connected, not just what values they have.

Good at:

- "Which components were removed from this aircraft?"
- "What maintenance events happened after that flight?"
- "Show me the path from aircraft to sensor"

The Supervisor: Bringing It Together

The **supervisor** is an agent that sits above the other two and routes questions to the right place.

```
                    User Question
                         |
                         v
                 ┌─── Supervisor ───┐
                 |                  |
                 v                  v
        Genie Space Agent    Neo4j MCP Agent
        (Lakehouse / SQL)    (Graph / Cypher)
```

It decides based on the nature of the question:

- **Numbers and trends** → Genie Space Agent
- **Relationships and structure** → Neo4j MCP Agent
- **Both needed** → Calls each agent in sequence, then combines the answers

Multi-Agent Routing in Action

"What is the average EGT for engine AC5?"

→ Supervisor sends to the **Genie agent** — this is a numeric aggregation over sensor data

"Which components were serviced on aircraft N95040A?"

→ Supervisor sends to the **Neo4j agent** — this is a relationship traversal through the graph

"Find aircraft with high vibration readings and show their maintenance history"

→ Supervisor calls **both agents in sequence**:

1. Genie agent identifies which aircraft have high vibration
2. Neo4j agent retrieves maintenance history for those aircraft
3. Supervisor synthesizes a combined answer

No Cypher or SQL knowledge required from the end user.

Alternative Architecture: Agent as a Serving Endpoint

There is another way to build this. Instead of connecting Neo4j as an MCP server at the supervisor level, you can **deploy the Neo4j agent itself as a Databricks Model Serving endpoint**.

How it works:

- The Neo4j agent is a standalone application — it has its own tools for searching products, traversing relationships, and managing conversation memory
- You package the agent with MLflow, register it in Unity Catalog, and deploy it to a serving endpoint
- Once deployed, it runs as an always-available API that scales automatically

The key difference: The agent's connection to Neo4j is baked into the deployment, not configured through MCP at the supervisor level. This gives the agent more control over how it queries the graph and what tools it exposes.

Combining the Deployed Agent with Genie

Once the Neo4j agent is running as a serving endpoint, you combine it with a Genie Space agent using the **Databricks AgentBricks supervisor** — built entirely through the workspace UI, no custom orchestration code required.

```
                    User Question
                         |
                         v
            ┌─── AgentBricks Supervisor ───┐
            |            (no code)         |
            v                              v
   Neo4j Agent Endpoint          Genie Space Agent
   (deployed via MLflow)         (Lakehouse / SQL)
```

Why choose this approach:

- The Neo4j agent can carry its own **conversation memory** — remembering user preferences across sessions

What Each Platform Brings

	Databricks	Neo4j
Stores	Tables and files	Nodes and relationships
Answers	"How much?" and "How often?"	"How is this connected?" and "What is affected?"
AI capability	Foundation Models, Genie	Vector indexes, GraphRAG, MCP
Strength	Scale, aggregation, ML	Relationships, traversal, pattern matching

The Spark Connector moves data. **MCP** lets agents query the graph directly. Together, the platforms stay connected at every layer.

Summary

Databricks + Neo4j is a natural pairing:

- The **Spark Connector** moves data between the Lakehouse and the Knowledge Graph
- **Lakehouse data becomes a graph** — making implicit relationships explicit and queryable
- **Neo4j as an MCP server** gives AI agents direct access to the knowledge graph
- **Multi-agent systems** route questions to the right platform automatically
- **GraphRAG** combines document search with graph context for smarter AI answers

Together, you get the analytical power of the Lakehouse **and** the relationship intelligence of the graph — connected, not siloed.

Appendix: Technical Details

Tables Become Graphs

Data in Databricks lives in **rows and columns**. Data in Neo4j lives as **nodes and relationships**.

The Spark Connector handles the translation:

Lakehouse (Databricks)	Knowledge Graph (Neo4j)
A row in an Aircraft table	An Aircraft node
A row in a Systems table	A System node
A foreign key linking them	A `HAS_SYSTEM` relationship

What was implicit in table joins becomes **explicit and traversable** in the graph.

The Neo4j Spark Connector

The **Neo4j Spark Connector** is how data flows between the two platforms.

Two-way bridge:

- **Databricks → Neo4j:** Take rows from your Lakehouse tables and turn them into nodes and relationships in a graph
- **Neo4j → Databricks:** Pull graph data back into DataFrames for analytics or ML

It works natively with Spark, so your existing Databricks notebooks and workflows can read from and write to Neo4j without leaving the platform.